Chapter 3
Hidden Markov Model

Abstract *: HMM is probabilistic model for machine learning. It is mostly used in speech recognition, to some extent it is also applied for classification task. HMM provides solution of three problems : evaluation, decoding and learning to find most likelihood classification. This chapter starts with description of Markov chain sequence labeler and then it follows elaboration of HMM, which is based on Markov chain. Discussion of all algorithms used to solve three basic problem of evaluation, decoding and learning is included. Chapter ends with enumerating the HMM.*

A sequence classifier or sequence labeler is a model whose job is to assign some label or class to each unit in a sequence. The HMM is probabilistic sequence classifiers; which means given a sequence of units (words, letters, morphemes, sentences etc) its job is to compute a probability distribution over possible labels and choose the best label sequence.

According to Han Shu [1997]Hidden Markov Models (HMMs) were initially developed in the 1960's by Baum and Eagon in the Institute for Defense Analyses.

3.1 Markov Chain

The Hidden Markov Model is one of the most important machine learning models in speech and language processing. In order to define it properly, we need to first introduce the Markov chain**.**

Markov chains and Hidden Markov Models are both extensions of the finite automata which is based on the input observations. According to Jurafsky, Martin [2005] a weighted finite-state automaton is a simple augmentation of the finite automaton in which each arc is associated with a probability, indicating how likely that path is to be taken. The sum of associated probabilities on all the arcs leaving a node must be 1.

According to Sajja [2012] a Markov chain is a special case of a weighted automaton in which the input sequence uniquely determines states the automaton will go through for that input sequence. Since they can't represent inherently ambiguous problems. Markov chain is only useful for assigning probabilities to unambiguous sequences.

A Markov chain is specified by the following components:

$Q = q_1 q_2 \ldots q_N$ a set of states

$A = [a_{ij}]_{NxN}$ a transition probability matrix A, each a_{ij} representing the probability of moving from state i to state j.

q_0, q_{end} a special start state and end state which are not associated with observations.

A Markov chain embodies an important assumption about these probabilities In a first-order Markov chain, the probability of a particular state is dependent only on the immediate previous state, Symbolically

Markov Assumption**:** $P(q_i|q_1 \ldots q_{i-1}) = P(q_i|q_{i-1})$

Since each a_{ij} expresses the probability $p(q_j|q_i)$, the laws of probability require that the values of the probabilities for some state must sum to 1.

$\pi = \pi_1, \pi_2, \ldots, \pi_N$ an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states.

Each π_i expresses the probability $p(q_i|START)$, all the π probabilities must sum to 1:

3.2 The Hidden Markov Model

A Markov chain is useful when we need to compute the probability for a sequence of events that we can observe in the world. It is useful even in events in which one is interested, but may not be directly observable in the world. For example for NER named entities tags are not observed in real world as we observe sequence of various weather cold, hot, rain. But sequence of words observed in real world and NER has to infer the correct named entities tags from these word sequence. The named entity tags are said to be hidden because they are not observed.

Jurafsky, Martin [2005] Hidden Markov Model (HMM) allows us to talk about both *observed* events (like words that we see in the input) and *hidden* events (like named entity tags) that we think of as causal factors in our probabilistic model.

3.2.1 Definition of HMM

According to Trujillo [1999], Jurafsky, Martin [2005] an HMM is specified by a set of states Q, a set of transition probabilities A, a set of observation likelihoods B, a defined start state and end state(s), and a set of observation symbols O, is not constructed from the state set Q which means observations may be disjoint from states. For example in NER sequence of words to be tagged are considered as observation where as States will be set of named entities classes :

Let's begin with a formal definition of a HMM, focusing on how it differs from a Markov chain. According to Jurafsky, Martin [2005] an HMM is specified by the following components:

$Q = q_1 q_2 \ldots q_N$	a set of states
$A = [a_{ij}]_{NxN}$	a transition probability matrix A, each a_{ij} representing the probability of moving from state i to state j,
$O = o_1 o_2 \ldots o_N$	a set of observations, each one drawn from a vocabulary $V = v_1, v_2, \ldots, v_V$.
$B = b_i(o_t)$	A set of observation likelihoods:, also called emission probabilities, each expressing the probability of an observation o_t being generated from a state i.
q_0, q_{end}	a special start state and end state which are not associated with observation.

An alternative representation for Markov chains is sometimes used for HMM, doesn't rely on a start or end state, instead representing the distribution over initial and accepting states explicitly:

$\pi = \pi_1, \pi_2, ..., \pi_N$ an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states.

$QA = \{q_x, q_y ...\}$ a set $QA \subset Q$ of legal accepting states

3.2.2 HMM Assumptions

Markov Assumption: A first-order Hidden Markov Model instantiates two simplifying assumptions. One: as in case of first-order Markov chain, the probability of a particular state is dependent only on the immediate previous state:

$$P(q_i|q_1 ... q_{i-1}) = P(q_i|q_{i-1})$$

Independent Assumption: The probability of an output observation o_i is dependent only on the state that produced the observation q_i, and not on any other state or any other observations:

$$P(o_i|q_1 \ldots q_i, \ldots, q_n, o_1, \ldots, o_i, \ldots, o_n) = P(o_i|q_i)$$

The Stationary Assumption Here it is assumed that state transition probabilities are independent of the actual time at which the transitions take place. Mathematically,

$$P(q_{t1+1} = j \mid q_{t1} = i) = P(q_{t2+1} = j|q_{t2} = i) \text{ for time } t_1 \text{ and } t_2$$

Having described the structure of an HMM, it will be logical to introduce the algorithms for computing things with them. An influential tutorial by Rabiner [1989], introduced the idea that Hidden Markov Models should be characterized by three fundamental problems: *Evaluation problem* can be used for isolated (word) recognition. *Decoding problem* is related to the continuous recognition as well as to the segmentation. *Learning problem* must be solved, if we want to train an HMM for the subsequent use of recognition tasks.

Evaluation: By evaluation it is meant what is the probability that a particular sequence of symbols is produced by a particular model? That is given an HMM $\lambda =$

(*A*,*B*) and an observation sequence *O*, to determine the likelihood *P*(*O*|λ). For evaluation following two algorithms are specified : the *forward algorithm* or the *backwards algorithm*, it may be noted this does not mean the forward-backward algorithm.

3.2.3 Computing Likelihood: The Forward Algorithm

Our first problem is to compute the likelihood of a particular observation sequence, That is a new observation sequence and a set of models. Thus the problem is to find a model which explains best the sequence, or in other terms which model gives the highest likelihood to the data.

*Computing Likelihood***:** Given an HMM λ= (*A*,*B*) and an observation sequence *O*, determine the likelihood *P*(*O*|λ).

In Hidden Markov Models, each hidden state produces only a single observation. Thus the sequence of hidden states and the sequence of observations have the same length. For example if there are *n* words (Observation) in a sentence then there will be *n* tags (hidden state).

Given this one-to-one mapping, and the Markov assumptions for a particular hidden state sequence $Q=q_0,q_1,q_2, ...,q_n$ and an observation sequence $O = o_1,o_2, ...,o_n$, the likelihood of the observation sequence (using a special start state q_0 rather than π probabilities) is:

$$P(O|Q) = \prod P(o_i|q_i) \times \prod P(q_i|q_{i-1}) \quad \textit{for } i = 1 \textit{ to } n$$

The computation of the forward probability for our observation (*Ram lives in Jaipur*) from one possible hidden state sequence PER NAN NAN CITY is as follows-

$$P(\text{Ram lines in Jaipur}|\text{PERNANNANCITY}) = P(\text{PER}|\text{start}) \times P(\text{NAN}|\text{PER}) \times P(\text{NAN}|\text{NAN}) \times P(\text{CITY}|\text{NAN}) \times P(\text{Ram}|\text{PER}) \times P(\text{lives}|\text{NAN}) \times P(\text{in}|\text{NAN}) \times P(\text{Jaipur}|\text{CITY})$$

In order to compute the true total likelihood of *Ram lives in Jaipur* however, we need to sum over all possible hidden state sequences. For an HMM with *N* hidden states

and an observation sequence of T observations, there are N^T possible hidden sequences. For real tasks, where N and T are both large, N^T is a very large number, and so one can not compute the total observation likelihood by computing a separate observation likelihood for each hidden state sequence and then summing them up.

Instead of using such an extremely exponential algorithm, we use an efficient algorithm called the forward algorithm. The forward algorithm is a kind of dynamic programming algorithm, The forward algorithm computes the observation probability by summing over the probabilities of all possible hidden-state paths that could generate the observation sequence.

Decoding : decoding means determining the most likely sequence of states that produced the sequence. Thus given an observation sequence O and an HMM $\lambda = (A,B)$, discover the best hidden state sequence Q. For this problem we use the *Viterbi algorithm*.

3.2.4 Decoding: The Viterbi Algorithm

For any model, such as an HMM, that contains hidden variables, the task of determining which sequence of variables is the underlying source of some sequence of observations is called the decoding task. The task of the decoder is to find the best hidden named entity tag sequence. More formally, given as input an HMM $\lambda = (A,B)$ and a sequence of observations $O = o_1,o_2, ...,o_T$, find the most probable sequence of states $Q = q_1q_2q_3 \ldots q_T$.

The following procedure is proposed to find the best sequence: for each possible hidden state sequence (PERLOCNANNAN, NANPERNANLOC, etc.), one could run the forward algorithm and compute the likelihood of the observation sequence for the given hidden state sequence.

Then one could choose the hidden state sequence with the max observation likelihood. But this can not be done since there are an exponentially large number of

state sequences. Instead, the most common decoding algorithms for HMMs is the Viterbi algorithm.

Like the forward algorithm it estimates the probability that the HMM is in state *j* after seeing the first *t* observations and passing through the most likely state sequence $q_1...q_{t-1}$, and given the automaton λ.

Each cell of the vector, v_t (*j*) represents the probability that the HMM is in state *j* after seeing the first *t* observations and passing through the most likely state sequence $q_1...q_{t-1}$, given the automaton λ. The value of each cell $v_t(j)$ is computed recursively taking the most probable path that could lead us to this cell. Formally, each cell expresses the following probability:

$$v_t(j) = P(q_0,q_1...q_{t-1},o_1,o_2 \ldots o_t, q_t = j|\lambda)$$

$$v_t(j) = \max \quad v_{t-1}(i)\, a_{ij}\, b_j(o_t) \qquad 1 \le i \le N-1$$

The three factors that are present in above equation for extending the previous paths to compute the Viterbi probability at time *t* are:

$v_{t-1}(i)$	the previous Viterbi path probability from the previous time step
a_{ij}	the transition probability from previous state q_i to current state q_j
$b_j(o_t)$	the state observation likelihood of the observation symbol o_t given the current state *j*.

Viterbi algorithm is identical to the forward algorithm exceptions. It takes the **max** over the previous path probabilities where as the forward algorithm takes the **sum**. Backpointers component is only component present in Viterbi algorithm that forward algorithm lacks. This is because while the forward algorithm needs to produce observation likelihood, the Viterbi algorithm must produce a probability and also the most likely state sequence. This best state sequence is computed by keeping track of the path of hidden states that led to each state.

3.2.5 Learning in HMM

Learning Generally, the learning problem is the adjustment of the HMM parameters, so that the given set of observations (called the *training set*) is represented by the model in the best way for the intended application. More explicitly given an observation sequence *O* and the set of states in the HMM, learn the HMM parameters *A* and *B*. Thus it would be clear that the ``quantity" which is to be optimized during the learning process can be different from application to application. In other words there may be several *optimization criteria* for learning, out of them a suitable one is selected depending on the application.

In essence it is to find the model that best fits the data. For obtaining the desired model the following 3 algorithms are applied:

- MLE (maximum likelihood estimation)
- Viterbi training(different from Viterbi decoding)
- Baum Welch = forward-backward algorithm

3.3 Advantages and Disadvantages of HMM

- The underlying theoretical basis is much more sound, elegant and easy to understand.
- It is easier to implement and analyze.
- HMM taggers are very simple to train (just need to compile counts from the training corpus).
- Performs relatively well (over 90% performance on named entities).
- Statisticians are comfortable with the theoretical base of HMM.
- Liberty to manipulate the training and verification processes.
- Mathematical / theoretical analysis of the results and processes.
- Incorporates prior knowledge into the architecture with good design.
- Initialize the model close to something believed to be correct.
- It eliminates label bias problem.

- It has also been proved effective for a number of other tasks, such as speech recognition, handwriting recognition and sign language recognition.
- Because each HMM uses only positive data, they scale well; since new words can be added without affecting learnt HMMs.

Disadvantages :

- In order to define joint probability over observation and label sequence HMM needs to enumerate all possible observation sequences.
- Main difficulty is modeling probability of assigning a tag to word can be very difficult if "words" are complex.
- It is not practical to represent multiple overlapping features and long term dependencies.
- Number of parameters to be evaluated is huge. So it needs a large data set for training.
- It requires huge amount of training in order to obtain better results.
- HMMs only use positive data to train. In other words, HMM training involves maximizing the observed probabilities for examples belonging to a class. But it does not minimize the probability of observation of instances from other classes.
- It adopts the Markovian assumption: that the emission and the transition probabilities depend only on the current state, which does not map well to many real-world domains;